МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
<БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ>

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №4

Специальность АС-67

Выполнил
Д. А. Ерш,
студент группы АС-67

Проверил
А. А. Крощенко,
ст. преп. кафедры ИИТ,
«___> _____2026 г.

Брест 2026

Цель: построить, обучить и оценить многослойный перцептрон (MLP) для решения задачи классификации

Вариант 4

Вариант 4 Распознавание рукописных цифр

- Digits

Основы машинного обучения, 2025, Крощенко А.А.

- **Задача:** распознать цифру от 0 до 9 (10 классов).
- **Архитектура:**
  - входной слой;
  - один скрытый слой с 32 нейронами (ReLU);
  - выходной слой с 10 нейронами (Softmax).
- **Эксперимент:** сравните результаты с архитектурой, где два скрытых слоя по 32 нейрона. Улучшилась ли точность?

Выполнение:

Код программы

```
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim

from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score

torch.manual_seed(42)
np.random.seed(42)

print("   Запуск задачи классификации (Digits)")

digits = load_digits()

X = digits.data
y = digits.target

print("Количество записей:", len(X))
print("Количество признаков:", X.shape[1])
print("Количество классов:", len(np.unique(y)))

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

X_train = torch.tensor(X_train, dtype=torch.float32)
X_test = torch.tensor(X_test, dtype=torch.float32)
```

```
y_train = torch.tensor(y_train, dtype=torch.long)
y_test = torch.tensor(y_test, dtype=torch.long)

class MLP(nn.Module):
    def __init__(self, input_size, second_layer=False):
        super().__init__()

        if second_layer:
            self.layers = nn.Sequential(
                nn.Linear(input_size, 32),
                nn.ReLU(),
                nn.Linear(32, 32),
                nn.ReLU(),
                nn.Linear(32, 10)
            )
        else:
            self.layers = nn.Sequential(
                nn.Linear(input_size, 32),
                nn.ReLU(),
                nn.Linear(32, 10)
            )

    def forward(self, x):
        return self.layers(x)

def train_model(second_layer, name):
    torch.manual_seed(42)

    model = MLP(X_train.shape[1], second_layer)

    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=0.001)

    for epoch in range(1, 201):
        model.train()

        output = model(X_train)
        loss = criterion(output, y_train)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        if epoch % 50 == 0:
            print(f"{name} | Эпоха {epoch}, Loss: {loss.item():.4f}")

    model.eval()

    with torch.no_grad():
        output = model(X_test)
        probabilities = torch.softmax(output, dim=1)
        predictions = torch.argmax(probabilities, dim=1).numpy()

    actual = y_test.numpy()

    accuracy = accuracy_score(actual, predictions)

    print(f"\n{name}")
    print(f"Accuracy: {accuracy:.4f}")

    return accuracy

print("\nОБУЧЕНИЕ МОДЕЛИ С ОДНИМ СКРЫТЫМ СЛОЕМ")
accuracy_1 = train_model(
    False,
    "MLP с одним скрытым слоем"
)
```

```
print("\nОБУЧЕНИЕ МОДЕЛИ С ДВУМЯ СКРЫТЫМИ СЛОЯМИ")
accuracy_2 = train_model(
    True,
    "MLP с двумя скрытыми слоями"
)

print("\nСРАВНЕНИЕ РЕЗУЛЬТАТОВ")
print(f"Один скрытый слой:  {accuracy_1:.4f}")
print(f"Два скрытых слоя:   {accuracy_2:.4f}")

change = accuracy_2 - accuracy_1

print(f"Изменение Accuracy: {change:+.4f}")

if accuracy_2 > accuracy_1:
    print("Добавление второго скрытого слоя улучшило точность.")
elif accuracy_2 < accuracy_1:
    print("Добавление второго скрытого слоя снизило точность.")
else:
    print("Точность моделей не изменилась.")
```

Результат:

Запуск задачи классификации (Digits)
Количество записей: 1797
Количество признаков: 64
Количество классов: 10

ОБУЧЕНИЕ МОДЕЛИ С ОДНИМ СКРЫТЫМ СЛОЕМ
MLP с одним скрытым слоем | Эпоха 50, Loss: 1.3899
MLP с одним скрытым слоем | Эпоха 100, Loss: 0.6066
MLP с одним скрытым слоем | Эпоха 150, Loss: 0.3103
MLP с одним скрытым слоем | Эпоха 200, Loss: 0.1948

MLP с одним скрытым слоем
Accuracy: 0.9417

ОБУЧЕНИЕ МОДЕЛИ С ДВУМЯ СКРЫТЫМИ СЛОЯМИ
MLP с двумя скрытыми слоями | Эпоха 50, Loss: 1.4050
MLP с двумя скрытыми слоями | Эпоха 100, Loss: 0.4706
MLP с двумя скрытыми слоями | Эпоха 150, Loss: 0.1978
MLP с двумя скрытыми слоями | Эпоха 200, Loss: 0.0946

MLP с двумя скрытыми слоями
Accuracy: 0.9556

СРАВНЕНИЕ РЕЗУЛЬТАТОВ
Один скрытый слой: 0.9417
Два скрытых слоя: 0.9556
Изменение Accuracy: +0.0139
Добавление второго скрытого слоя улучшило точность.

Вывод: В ходе лабораторной работы была разработана и обучена нейронная сеть многослойный перцептрон (MLP) для распознавания рукописных цифр от 0 до 9. Были реализованы и сравнены две архитектуры: с одним скрытым слоем из 32 нейронов и с двумя скрытыми слоями по 32 нейрона. В результате эксперимента первая модель показала точность 94,17%, а вторая — 95,56%. Таким образом, добавление второго скрытого слоя позволило повысить точность классификации на

1,39 процентного пункта. Следовательно, для данной задачи архитектура с двумя скрытыми слоями показала лучший результат.